

Final Project: Visual Understanding with TextVQA

Introduction

Visual question answering (VQA) is a multimodal task that requires a model to answer natural language questions about the content of an image. While early VQA benchmarks focused on object recognition and scene understanding, the TextVQA dataset poses a more demanding challenge: questions can be answered only by reading and reasoning about text embedded within the image itself [1]. Examples include identifying a brand name printed on a product, reading a number displayed on a scoreboard, or interpreting a label on a shelf. This requirement for optical character recognition (OCR) combined with language understanding makes TextVQA a rigorous testbed for evaluating real-world visual comprehension.

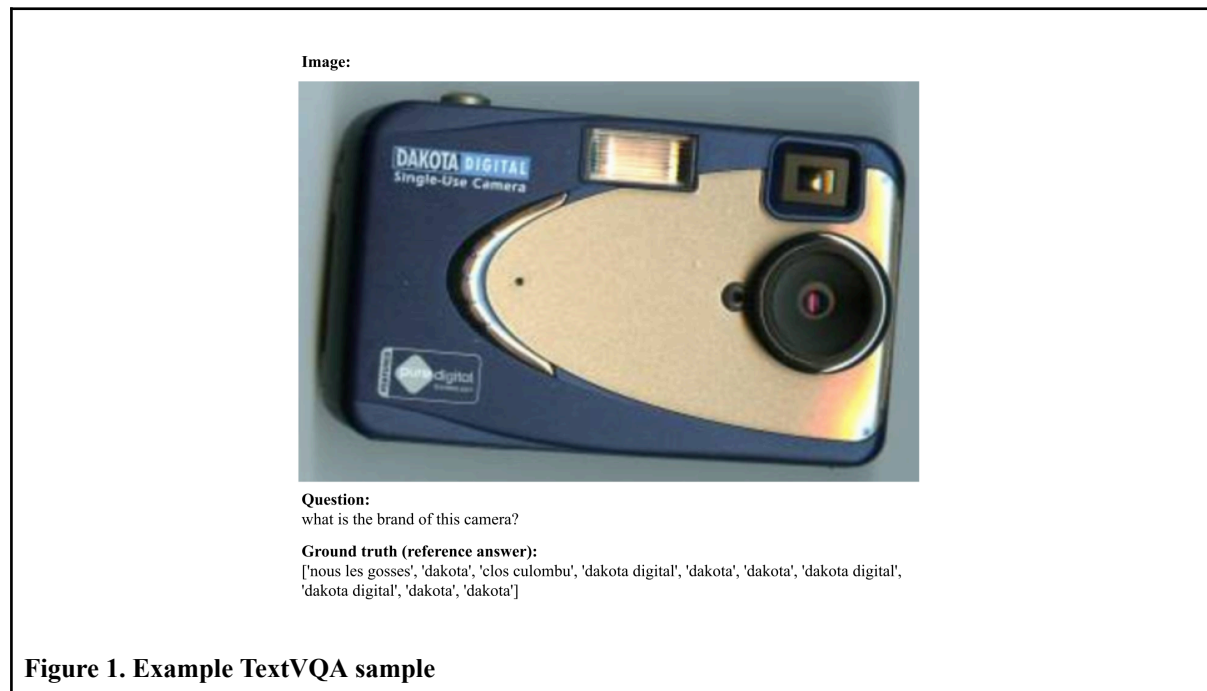
Recent advances in vision-language models (VLMs) have significantly expanded the limits of multimodal understanding. Architectures such as BLIP-2 [2] and LLaVA [3] introduced efficient mechanisms for bridging visual encoders and large language models, which allow zero-shot instruction following on image-based tasks. More recently, models such as Qwen2.5-VL [4] and InternVL2.5 [5] have pushed further by incorporating dynamic resolution processing and fine-grained spatial reasoning, which make them especially well-suited for document-level and text-in-image understanding. Despite these improvements, performance on TextVQA is still sensitive to factors such as image resolution, text legibility, and the specificity of the prompting strategy, which are not fully understood yet.

Therefore, this project evaluates Qwen2.5-VL-3B, a 3-billion-parameter open-source VLM [1], on the TextVQA validation set under zero-shot and prompt-engineered conditions. Specifically, there are three main goals for this study. First, we want to establish a zero-shot baseline that captures the model's default behavior without task-specific instructions. Second, we aim to systematically explore prompt engineering strategies that improve answer quality without modifying model parameters. Lastly, we try to analyze both quantitative metrics and qualitative examples to identify where the model succeeds and where it fails. By examining the gap between semantic understanding, measured by LLM-as-a-Judge and lexical accuracy, measured by exact match and VQA accuracy, we aim to characterize the specific challenges that text-in-image tasks pose for current VLMs and to offer practical guidance for prompt design in this setting.

Methods

Dataset Description

This project uses the TextVQA dataset accessed through the Hugging Face Datasets library. The dataset consists of images paired with natural language questions that require reading text within the image to answer correctly. Each sample includes an image (in JPEG format), a question, and a set of ground-truth answers (Fig. 1). The dataset is pre-split into training, validation, and test sets. In this study, all analyses were performed on the validation set.



VLM Model Choice

Qwen2.5-VL-3B is a multilingual vision–language model (VLM) accessed via Hugging Face (Qwen/Qwen2.5-VL-3B-Instruct), containing approximately 3 billion parameters. We selected this model over other candidates, including BLIP-2 (2.7B), LLaVA-Phi-3-mini (3.8B), and InternVL2.5 for three reasons. First, Qwen2.5-VL-3B is specifically designed with enhanced OCR and document understanding capabilities, making it well-suited for TextVQA, which demands accurate text reading within natural images. Second, its dynamic resolution processing allows it to handle images of varying sizes while preserving spatial detail, which is an important property given the diversity of image types in TextVQA. Third, at approximately 3 billion parameters, it offers a favorable trade-off between inference efficiency and task performance, enabling evaluation across the full validation set without prohibitive computational cost. These features enable Qwen2.5-VL-3B to capture fine-grained spatial information and support detailed visual reasoning across complex image-based inputs [4].

Normalization Method

Prior to evaluation, both predicted and reference answers were normalized using the following procedure. All text was converted to lowercase. Punctuation was handled selectively: apostrophes (') were preserved, while colons (:) and decimal points (.) were retained only when they appeared between digits (e.g., 10:04, 123.456) and replaced with spaces otherwise. All remaining punctuation and special symbols were replaced with spaces. The text was then tokenized, and article words (a, an, the) were removed. Finally, extra whitespace was collapsed into a single space. This normalization was applied consistently across all prompt configurations and both predicted and reference texts to ensure fair comparison.

*Model Evaluation Metrics*I. Quantitative EvaluationPrimary metrics: visual question answering (VQA) accuracy and exact match (EM)

To evaluate model performance, we employed two categories of evaluation metrics. The primary metric was VQA accuracy and EM, computed on the validation set to measure the proportion of correctly answered questions. The primary metrics were performed on comparing the normalized predicted and reference texts to ensure consistency. Both VQA accuracy and EM rely on exact string matching after normalization, but they differ in how matches are scored.

VQA accuracy measures agreement between the predicted and multiple human-annotated reference answers, allowing for partial credit [6]. It is computed as the number of reference answers that match the prediction divided by three, and capped at one [7]:

$$Accuracy = \min\left(\frac{\text{Number of matching answers}}{3}, 1\right)$$

The division by three reflects the evaluation design in which each question is annotated by ten human responses, and a prediction is considered fully correct if at least three annotators provide the same answer. This formulation accounts for variability and ambiguity in human annotations while still rewarding consensus.

In contrast, EM is a binary metric that assigns a score of 1 if the predicted answer exactly matches any one of the reference answers, and 0 otherwise.

Secondary metrics: BLEU, METEOR, ROUGE-1F, and LLM-as-a-Judge

Secondary metrics captured aspects of semantic similarity that exact matching cannot. BLEU measures lexical similarity by computing n-gram precision between predicted and reference texts, with a brevity penalty applied when the prediction is substantially shorter than the reference [4]. In the VQA context, this brevity penalty is particularly relevant because ground truth answers are typically one to three words long, which means that any over-generation is penalized. METEOR extends beyond exact word matching by incorporating synonym matches, stemming, and word order, combining precision and recall into a weighted harmonic mean [4]. This makes it especially useful in VQA settings where the model may predict a semantically correct but lexically different answer. For example, a model predicting “twenty-two” when the reference answer is “22” would receive no credit under exact match, but METEOR would recognize the two as equivalent. ROUGE-1F measures the overlap of unigrams between predicted and reference texts. It is computed as the harmonic mean (F1 score) of precision and recall, where precision reflects the proportion of overlapping words in the generated text, and recall reflects the proportion of overlapping words in the reference text [8].

LLM-as-a-Judge was implemented using the Mistral-7B-Instruct model (mistralai/Mistral-7B-Instruct-v0.2) to assess the semantic correctness of predicted answers. The model was selected due to its open-source availability and strong instruction-following capabilities, which are critical for prompt-based evaluation tasks such as semantic answer judging [5,6]. Its relatively compact size (7 billion parameters) enables efficient local inference without reliance on external APIs, supporting cost-effective and reproducible evaluation. Importantly, using a separate judge model rather than the same Qwen2.5-VL-3B used for prediction avoids potential self-evaluation bias, in which a model may unduly favor its own output style. Given a question, ground truth answers, a prediction, and specific grading criteria, these inputs are formatted into a structured prompt and provided to the model, which assigns a binary score (1 = correct, 0 = incorrect) based on semantic equivalence, accounting for synonyms and variations in phrasing. This method complements traditional exact match and VQA accuracy metrics by capturing semantically correct responses that may differ lexically from the reference.

II. Qualitative Evaluation

Qualitative evaluation was conducted to examine model behavior across different image-question pairs and to identify patterns of success and failure under baseline and prompt-optimized settings. Unlike quantitative evaluation, which summarizes performance using aggregate metrics, qualitative analysis focuses on understanding how and why models succeed or fail on specific examples.

A subset of validation samples was selected, and model predictions were compared against ground truth answers. Performance was assessed using both EM and LLM-as-a-Judge scores. EM was not evaluated for the baseline model, as its outputs were generated as full sentences and did not match the short-answer format of the reference annotations. In contrast, LLM-as-a-Judge evaluation was applied to all models, as it allows for semantic matching and is less sensitive to formatting differences.

Examples of best-performing cases were defined as those in which the model produced correct or semantically equivalent answers (EM = 1 or LLM-as-a-Judge = 1). These cases were used to illustrate the model's ability to correctly extract textual information (e.g., brand names, numbers, and labels) and to align with human annotations.

Best-performing cases were defined as those in which all prompt-engineered models (Prompts 1-7) produced correct answers (EM = 1), and the LLM-as-a-Judge also assigned a score of 1. These cases illustrate the model's ability to correctly extract textual information such as brand names, numbers, and labels, and to align with human annotations.

Worst-performing cases were defined as those in which the model received EM = 0 and LLM-as-a-Judge = 0 across the baseline and all prompt configurations. Because these errors persist regardless of the prompting strategy, they are indicative of perceptual or reasoning limitations intrinsic to the model rather than issues addressable through prompt design.

Common error types identified in this analysis included OCR failure on small or occluded text, hallucination of plausible but unsupported answers, and failures in spatial counting tasks involving visually cluttered scenes.

Experimental Design

Study Design

This study follows a three-stage evaluation framework: zero-shot evaluation, prompt engineering, and result analysis. In the zero-shot setting, the pretrained model is applied directly to the TextVQA validation set without any task-specific adaptation, establishing a baseline that reflects default model behavior. Prompt engineering is then used to improve performance by designing and evaluating multiple prompt configurations without modifying model parameters. The prompt configurations are designed as a progressive ablation: each prompt adds one additional constraint on top of the previous, enabling us to isolate the contribution of each component, including brevity, OCR awareness, uncertainty handling, and output format, to overall performance. Finally, the result analysis compares zero-shot and prompt-engineered performance using both quantitative metrics and qualitative examples, with a focus on identifying common error types, including OCR misinterpretation and reasoning failures.

Prompt Used for Inference

For each sample, the input is formatted as a user message containing the image and a textual instruction. We evaluated a total of seven prompt configurations, including one baseline condition with no explicit instruction (Table 1). The configurations are organized into two groups based on their design approach. Group A (Prompts 1-4) follows a progressive ablation structure, where each prompt builds on the previous by adding one additional constraint, allowing us to isolate the contribution of each component, including brevity, text recognition guidance, uncertainty handling, and output format constraints, to overall performance. Group B (Prompts 5-7) tests more advanced multi-stage strategies that go beyond single-instruction prompting, including OCR token guidance, two-stage text extraction, and two-pass answer verification. Together, these two groups allow us to compare the effect of incremental instruction refinement against more structurally complex prompting approaches.

<i>Group A: Single-stage prompts with progressive constraints (Baseline, Prompts 1-4)</i>		
Setting	Prompt Description	Component
Baseline	No explicit prompt (model default behavior).	None
Prompt 1	(1) Answer the question based on the image. (2) Use a short answer only.	Brevity constraint
Prompt 2	(1) Answer the question based on the image. (2) Read any visible text carefully. (3) Use a short, simple answer only.	Brevity constraint + Text recognition guidance

	(4) Do not include explanations.	
Prompt 3	(1) Answer the question based on the image. (2) If text is visible, copy it directly. (3) If unsure, respond “unanswerable”. (4) Use a short, simple answer only. (5) Do not include explanations.	Brevity constraint + Text recognition guidance + Uncertainty handling
Prompt 4	(1) Answer the question based on the image. (2) If text is visible, copy it directly. (3) For numeric answers, use digits (e.g., 10 instead of ten). (4) For text answers, return only English alphabet characters. (5) Use a short, simple answer only. (6) Do not include explanations.	Brevity constraint + Text recognition guidance + Uncertainty handling + Output format constraints
<i>Group B: Advanced multi-stage prompts (Prompts 5-7)</i>		
Setting	Prompt description	strategy
Prompt 5	(1) Answer the question using OCR tokens. (2) Return a short, exact answer.	Brevity constraint + OCR token guidance
Prompt 6	(1) Two-stage prompting: a. Extract all visible text from the image b. Answer the question using the extracted text. (2) Return a short, exact answer. (3) Do not include explanations.	Brevity constraint + Text extraction guidance + Two-stage prompting
Prompt 7	(1) Two-pass prompting with answer verification. a. Generate an initial answer using OCR tokens b. Verify, refine, and shorten the initial answer using visible text and the question. (2) Return a final answer.	Brevity constraint + OCR token guidance + Two-pass prompting with answer verification
Table 1. Prompt configurations used for VQA evaluation. Summary of the prompt configurations used in this study. The prompts progressively incorporate additional constraints, including short answer format, text recognition, and uncertainty handling, as well as more advanced strategies such as OCR token guidance, multi-stage text extraction, and answer refinement. Group A (Prompts 1-4) follows a progressive single-stage design in which each prompt adds one constraint to the previous one. Group B (Prompts 5-7) introduces advanced multi-stage strategies.		

The prompt is formatted using the model's chat template, which appends a generation token to indicate that the model should produce a response. The image and text are jointly encoded and passed to the model. Inference is performed using greedy decoding, which ensures deterministic and reproducible outputs. A maximum generation length of 16 tokens was selected because ground-truth answers in TextVQA are typically one to three words long; this constraint prevents over-generation while leaving sufficient capacity for all plausible answers. Padding was set to true to ensure predictions are of consistent length across batches.

Results

Quantitative Evaluation

As shown in Table 2, prompt engineering substantially improved model performance across both primary and secondary metrics compared to the baseline. The baseline exhibited near-zero performance on the primary metrics (VQA accuracy = 0.0002, EM = 0.0006) and the lowest scores across most secondary metrics (BLEU = 0.0117, METEOR = 0.2204, ROUGE-1F = 0.1600). Notably, however, the baseline achieved a moderately high LLM-as-a-Judge score of 0.7618, in striking contrast to its near-zero exact-match rate. This gap reveals that the baseline model frequently produced semantically correct content, but in a sentence-style format that was entirely misaligned with the short-answer annotations.

Among Group A prompts, performance ranked as Prompt 1, Prompt 2, Prompt 4, and Prompt 3. Prompt 1 performed best overall, with VQA accuracy of 0.7632, EM of 0.8110, BLEU of 0.1733, ROUGE-1F of 0.6657, and LLM-as-a-Judge score of 0.8374. This shows that simply asking the model to give a short answer was enough to greatly improve performance over the baseline. In contrast, adding more instructions did not further improve the results. Prompt 4, which had the most constraints, performed worse than Prompt 1 in both BLEU and VQA accuracy, suggesting that too many instructions may make the answer less aligned with the ground-truth format. Among Group B prompts, performance ranked as Prompt 5, Prompt 7, and Prompt 6. Prompt 5, which asked the model to use OCR tokens, had the best Group B performance, with VQA accuracy of 0.7445 and EM of 0.7920. However, it was still lower than Prompt 1. The more complex two-stage strategies in Prompts 6 and 7 did not show clear improvement over simpler prompts. Across all prompts, METEOR scores were relatively stable, ranging from 0.3914 to 0.4067, suggesting that the general semantic content of the answers was similar even when prompt complexity changed.

Evaluation Metric	Primary Metrics		Secondary Metrics			
	VQA Accuracy	EM	BLEU	METEOR	ROUGE-F1	LLM-as-a-Judge Score
Baseline	0.0002	0.0006	0.0117	0.2204	0.1600	0.7618
Prompt 1	0.7632	0.8110	0.1733	0.4064	0.6657	0.8374
Prompt 2	0.7472	0.7978	0.1354	0.4067	0.6583	0.8326
Prompt 3	0.7223	0.7740	0.1245	0.4015	0.6460	0.823
Prompt 4	0.7367	0.7852	0.1185	0.4037	0.6531	0.8272
Prompt 5	0.7445	0.7920	0.1570	0.3927	0.6501	0.814
Prompt 6	0.7387	0.7848	0.1248	0.3984	0.6512	0.818
Prompt 7	0.7424	0.7904	0.1530	0.3914	0.6476	0.8114

Table 2. Performance comparison across prompt configurations. Comparison of baseline and

prompt-engineered models across primary (VQA accuracy, EM) and secondary (BLEU, METEOR, ROUGE-F1, and LLM-as-a-Judge) evaluation metrics on the validation set.

Qualitative Evaluation

To complement the quantitative results and better understand the factors that contribute to both best and worst performance, a qualitative evaluation was conducted to investigate model behavior on representative examples. Worst-performing cases were defined as those with EM = 0 and LLM-as-a-Judge score = 0 across the baseline and all prompt configurations. In contrast, best-performing cases were defined as those with EM = 1 across all prompt configurations (excluding the baseline), and LLM-as-a-Judge score = 1. The baseline model was excluded from the EM criterion, as it generated longer, sentence-style responses that did not align with the short-answer format of the ground truth annotations.

Worst-performing examples reveal several common sources of error across both baseline and prompt-engineered models (Fig. 3). In Example (a), in response to the question “what football league is the jacket from on the man pointing?”, the models predicted either “premier league” or “football league” instead of the correct answers, including “ryman”, “the ryman football league”, “macron” and “ryman macron” (Fig. 3a, Table 3). Based on the image, the correct answer appears as small text on the back of the jacket, which likely made it difficult for the models to read accurately. Additionally, the text is partially obscured and embedded within a larger scene, which may have further confused the model and led it to predict more general or familiar league names. In Example (b), in response to the question “what is the time?”, the models predicted either “10:07” or “10:08” instead of the correct answers, including “5:41”, “5:40”, “5:42”, and “8:00” (Fig. 3b, Table 4). In the image, the watch is positioned at an angle that makes it difficult for the model to correctly interpret the orientation of the clock face. This likely led to systematic misreading of the time. Together, these examples suggest that the model struggles with small or partially obscured text and with interpreting rotated or non-standard visual orientations.

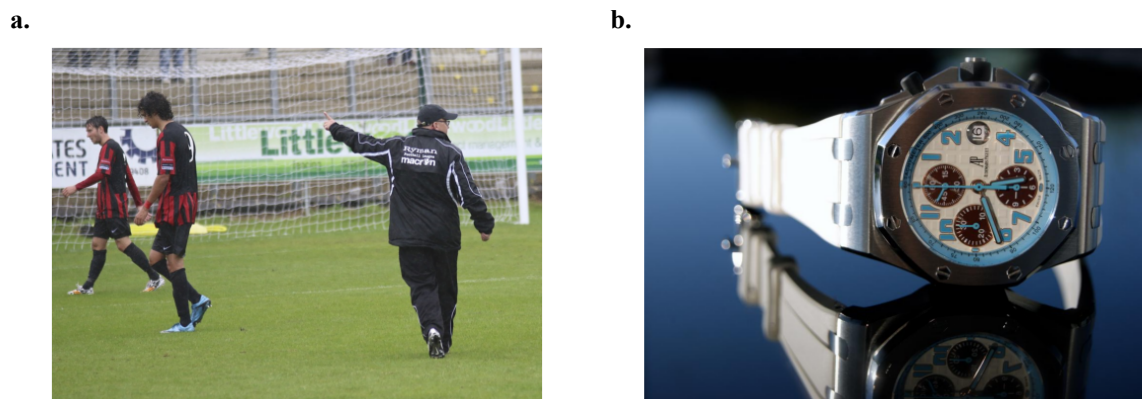


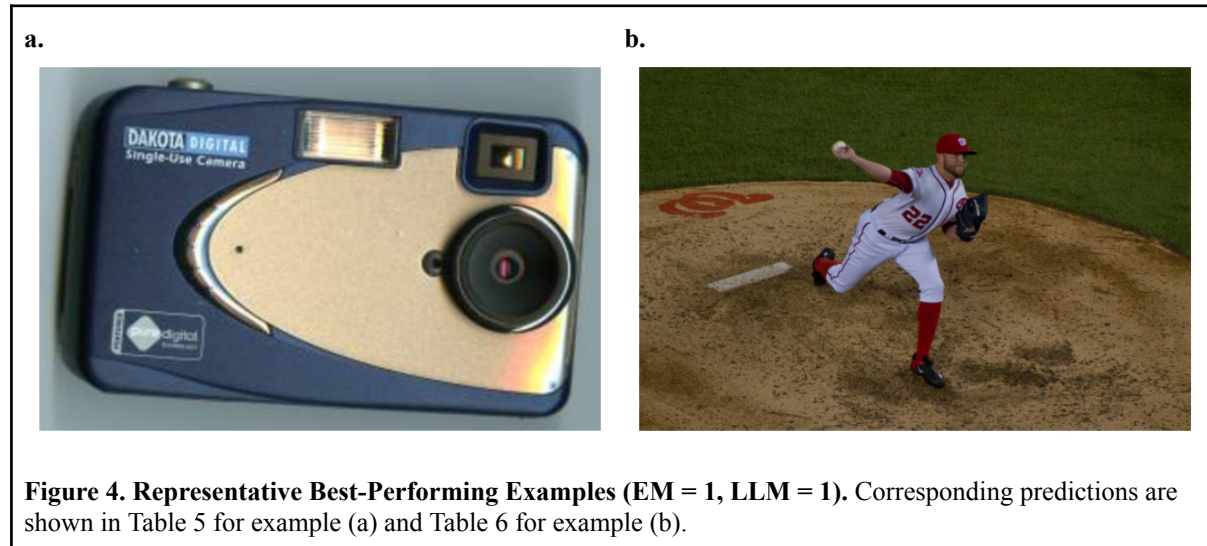
Figure 3. Representative Worst-Performing Examples (EM = 0, LLM = 0). Corresponding predictions are shown in Table 3 for example (a) and Table 4 for example (b).

Prompt Configuration	Prediction
Baseline	jacket in picture is from football league championship as indicated by
Prompt 1	premier league
Prompt 2	premier league
Prompt 3	premier league
Prompt 4	premier league
Prompt 5	premier league
Prompt 6	football league
Prompt 7	premier league
Table 3. Model Predictions for Example (a): Worst-Performing Case. Question: What football league is the jacket from on the man pointing? Ground Truth: ['ryman', 'the ryman football league', 'macron', 'ryman macron']	

Prompt Configuration	Prediction
Baseline	time shown on watch is 10:08.
Prompt 1	10:08
Prompt 2	10:07
Prompt 3	10:08
Prompt 4	10:08
Prompt 5	10:08
Prompt 6	10:08
Prompt 7	10:08
Table 4. Model Predictions for Example (b): Worst-Performing Case. Question: What is the time? Ground Truth: ['5:41', '8:00', '5:40', '5:42']	

Best-performing examples highlight consistent patterns of success across both baseline and prompt-engineered models (Fig. 4). In Example (a), in response to the question “what is the brand of this camera?”, the baseline model generated a full sentence response containing the key phrase “dakota digital”, while the prompt-engineered models produced concise answers such as “dakota digital” and “dakota” (Fig. 4a, Table 5). All predicted answers were semantically consistent with the ground truth, while prompt-engineered models additionally achieved exact matches. In the image, the camera displays clear and prominent text corresponding to the correct answer, which likely facilitated accurate text recognition. In Example (b), in response to the question “what number is on the player's jersey?”, the

baseline model again produced a full sentence response containing the correct number “22”, while all prompt-engineered models directly predicted “22”, matching the ground truth (Fig. 4b, Table 6). The correct answer is clearly visible on the player’s jersey, making it easy for the model to extract. Altogether, these examples demonstrate that the model performs best when relevant text is clearly visible and unambiguous, requiring minimal visual reasoning beyond direct text extraction.



Prompt Configuration	Prediction
Baseline	brand of camera in picture is dakota digital
Prompt 1	dakota digital
Prompt 2	dakota digital
Prompt 3	dakota
Prompt 4	dakota
Prompt 5	dakota
Prompt 6	dakota
Prompt 7	dakota

Table 5. Model Predictions for Example (a): Best-Performing Case.
Question: What is the brand of this camera?
Ground Truth: ['nous les gosses', 'dakota', 'clos culombu', 'dakota digital']

Prompt Configuration	Prediction
Baseline	number on player's jersey is 22
Prompt 1	22
Prompt 2	22
Prompt 3	22
Prompt 4	22
Prompt 5	22
Prompt 6	22
Prompt 7	22
Table 6. Model Predictions for Example (b): Best-Performing Case. Question: What number is on the player's jersey? Ground Truth: ['22']	

Discussion

Our results show that prompt engineering clearly improved VQA performance. Prompt 1, which only asked the model to give a short answer, performed best across most metrics, with VQA accuracy of 0.7632, EM of 0.8110, and LLM-as-a-Judge score of 0.8374. This suggests that a simple brevity instruction was enough to make the model outputs much more evaluable. In comparison, the baseline had a relatively high LLM-as-a-Judge score of 0.7618. This means that many answers were still semantically reasonable, but it scored near-zero on VQA accuracy and EM because it produced full-sentence answers rather than concise, ground-truth-style responses. This shows that the main issue was not necessarily image understanding, but whether the answer format matched the evaluation setting.

Interestingly, adding more detailed instructions, such as OCR-related guidance, uncertainty handling, or specific rules for digits versus text, did not consistently improve performance and sometimes slightly reduced it. Prompts 2 to 4 all performed worse than Prompt 1. This suggests that adding too many constraints may interfere with the model's natural response generation, either by creating competing goals or by making the output less aligned with the ground-truth answers. This is also consistent with the broader prompt engineering literature, which suggests that over-specifying instructions may not help when the model already has the needed ability.

The qualitative analysis also helped show when the model worked well and when it failed. The model performed well when the target text was large, clear, and near the center of the image, such as in the camera brand and jersey number examples. In these cases, simple OCR was enough to give the correct answer. The errors mainly happened in two situations. First, the model struggled with small or cluttered text, such as a year printed on a wall calendar with many surrounding objects. In these examples, it often produced reasonable-looking but

incorrect numbers, suggesting that the model may not consistently capture fine-grained text at lower image resolutions. Second, the model struggled with spatial counting tasks, such as counting price tags on a shelf. Here, it seemed to rely on an approximate guess instead of carefully counting each region. This suggests that current VLMs still have limitations when a task requires both detailed text reading and spatial reasoning at the same time.

Several additional limitations should also be noted. First, a small number of ground-truth questions had typos, such as “calender” instead of “calendar” or “How man” instead of “How many.” These errors may affect model performance on those samples for reasons unrelated to image understanding. Second, some baseline outputs were cut off by the 16-token generation limit. However, since the correct answer usually appeared near the beginning of the response, this likely did not strongly affect the LLM-as-a-Judge scores, though it may have slightly lowered the exact-match rate for the baseline.

Several directions could extend this work. First, Qwen2.5-VL-3B could be fine-tuned on the TextVQA training set using parameter-efficient methods such as LoRA [9], which may help improve performance on small or dense text. Second, future work could test whether adding OCR token inputs from the TextVQA annotations [1] improves performance by reducing the text-reading burden on the visual encoder. Third, the best prompt should also be evaluated on the held-out test set to better assess generalization. A more detailed error analysis by question type, such as brand identification, number reading, and spatial counting, could also show whether different prompts work better for different subtasks. Finally, comparing Qwen2.5-VL-3B with other lightweight VLMs, such as LLaVA-Phi-3-mini [10] and InternVL2.5 [4], under the same evaluation setting would help determine whether these findings are specific to this model or reflect broader patterns across similar VLMs.

Conclusion

This study evaluated Qwen2.5-VL-3B on the TextVQA benchmark using zero-shot and prompt-engineered settings. We found that the zero-shot baseline often produced semantically reasonable answers, but the responses were not well aligned with the short ground-truth format. As a result, exact match and VQA accuracy were near zero, even though the LLM-as-a-Judge score was moderately high. Adding a simple instruction to give only a short answer greatly improved performance, reaching a VQA accuracy of 0.7632 and an exact match rate of 0.8110. More complex prompts did not improve the results further, suggesting that too many instructions may not help when the model already has the needed ability.

The qualitative results showed that the model worked best when the target text was clear, large, and easy to read. It performed worse when the text was small, occluded, or cluttered, and when the task required precise counting or spatial localization. These limitations may be difficult to fix through prompt engineering alone and may require model improvement or task-specific fine-tuning.

Overall, these findings show both the potential and limitations of general-purpose VLMs for text-based VQA. Future work could explore LoRA fine-tuning on the TextVQA training set

SHBT 261

Group member: Diana Hou, Tzu-Hsi (Cece) Jen

Date: May 7th, 2026

or test whether adding OCR tokens alongside the image can improve performance on more challenging examples.

Appendix

This is the link to the dataset: [link](#)

This is the link to the code for this project: [link](#)

References

1. Singh A, Natarajan V, Shah M, Jiang Y, Chen X, Batra D, et al. Towards VQA models that can read. arXiv [cs.CL]. 2019. doi:[10.48550/arXiv.1904.08920](https://doi.org/10.48550/arXiv.1904.08920)
2. Li J, Li D, Savarese S, Hoi S. BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. arXiv [cs.CV]. 2023. doi:[10.48550/arXiv.2301.12597](https://doi.org/10.48550/arXiv.2301.12597)
3. Liu H, Li C, Wu Q, Lee YJ. Visual Instruction Tuning. arXiv [cs.CV]. 2023. doi:[10.48550/arXiv.2304.08485](https://doi.org/10.48550/arXiv.2304.08485)
4. Bai S, Chen K, Liu X, Wang J, Ge W, Song S, et al. Qwen2.5-VL Technical Report. arXiv [cs.CV]. 2025. doi:[10.48550/arXiv.2502.13923](https://doi.org/10.48550/arXiv.2502.13923)
5. Chen Z, Wu J, Wang W, Su W, Chen G, Xing S, et al. InternVL: Scaling up vision foundation models and aligning for generic visual-linguistic tasks. arXiv [cs.CV]. 2023. doi:[10.48550/arXiv.2312.14238](https://doi.org/10.48550/arXiv.2312.14238)
6. Improving Automatic VQA Evaluation Using Large Language Models. [cited 27 Apr 2026]. Available: <https://arxiv.org/html/2310.02567v2>
7. VQA: Visual Question Answering. [cited 27 Apr 2026]. Available: <https://visualqa.org/evaluation.html>
8. When Metrics Disagree: Automatic Similarity vs. LLM-as-a-Judge for Clinical Dialogue Evaluation. [cited 23 Apr 2026]. Available: <https://arxiv.org/html/2603.00314v2>
9. Hu EJ, Shen Y, Wallis P, Allen-Zhu Z, Li Y, Wang S, et al. LoRA: Low-Rank Adaptation of large language models. arXiv [cs.CL]. 2021. doi:[10.48550/arXiv.2106.09685](https://doi.org/10.48550/arXiv.2106.09685)
10. Zhu Y, Zhu M, Liu N, Ou Z, Mou X, Tang J. LLaVA-phi: Efficient multi-modal assistant with small language model. arXiv [cs.CV]. 2024. doi:[10.48550/arXiv.2401.02330](https://doi.org/10.48550/arXiv.2401.02330)